SpatialNet: Locating Pointed-At Objects from a Single Image with a 0.7M-Parameter CNN on Smart Glasses

ANONYMOUS AUTHOR(S)

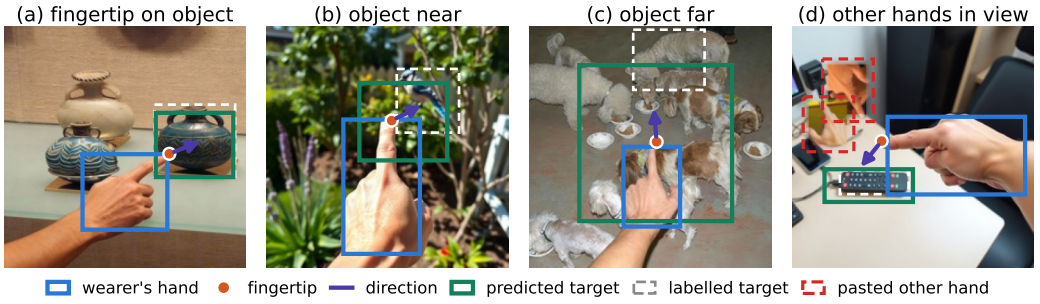


Fig. 1. SpatialNet's predictions from one frame of the glasses' camera: the wearer's hand, the fingertip, the pointing direction and the pointed-at object (white dashed: labelled object). Objects touched (a) and near (b) are found. The far dog among similar dogs (c) is a typical failure: the box is far too large. The model follows the wearer's hand when other hands are in view (d; these hands are synthetic, pasted as in Section 4.6). EgoPoint test images.

Pointing is the most direct way to ask smart glasses about something in view ("what is *that*?"). Before any assistant can answer, the glasses must work out which object the wearer is pointing at, from their own camera and quickly. Current pointing-grounding models are transformers and multimodal LLMs with hundreds of millions to billions of parameters. They run on GPUs, the LLMs decode their answers token by token, and some models need a text prompt or the hand box as input. We ask how far a model that fits on the glasses can go.

SpatialNet takes nothing but one 2D RGB frame: no text, no hand box, no depth, no external hand or pose model. In a single forward pass of a compact convolutional network without self-attention (0.72M parameters, 0.23 GFLOPs, 1.6 MB in FP16), it outputs the wearer's hand, the fingertip, the pointing direction and the box of the pointed-at object. To make this possible at that size, its heads form an instance-anchored chain: each stage reads a shared feature map where the previous stage predicted, so the hand, fingertip, direction and target stay on one hand.

On the EgoPoint test split, SpatialNet finds the wearer's hand with a P@0.5 of 92.6% (share of frames with IoU ≥ 0.5) and localises the pointed-at object with a P@0.5 of 58.9% (mIoU 53.3%; 58.9–59.0% across three training seeds). This is comparable to zero-shot 7–8B multimodal LLMs reported on the same split (44.5–49.8% P@0.5) and below models of 226M–7B parameters trained on EgoPoint (63.6–93.9%). It drops to 25.4% for objects far from the fingertip. Ablations and a resolution study show what matters at this scale:

- the anchoring keeps boxes and direction precise (without it, P@0.7 drops by 2.8 points and the direction error rises by 3.4°);
- input resolution is the largest lever (65.1% at 320 px with the same parameters);

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2474-9567/2026/10-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

- a learned prior from explicit ray geometry, which we also test, collapses into a fingertip-proximity prior on EgoPoint's touch-dominated data.

On an engineering unit of a pair of smart glasses, the 160 px model runs in 60–80 ms per frame, and running it continuously drains less battery than merely streaming frames to a phone.

CCS Concepts: • **Human-centered computing** → **Ubiquitous and mobile computing systems and tools**; **Gestural input**; • **Computing methodologies** → *Object detection.*

Additional Key Words and Phrases: smart glasses, pointing gesture, deictic reference, egocentric vision, on-device inference, efficient neural networks

ACM Reference Format:
Anonymous Author(s). 2026. SpatialNet: Locating Pointed-At Objects from a Single Image with a 0.7M-Parameter CNN on Smart Glasses. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 1, 1 (October 2026), 28 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Camera-equipped smart glasses are becoming everyday devices, and an assistant on them is most useful when the wearer can refer to things in view. Pointing is the most direct way to do so: "what is *that*?", "remind me to buy *this*". Systems have combined speech with pointing since Bolt's *Put-that-there* [4], and recent wearable assistants use pointing and gaze to resolve such pronouns [17]. Before any of this works on glasses, the device must answer a spatial question from its own camera: *which object is the wearer pointing at?*

Answering this on glasses is hard for three reasons.

- *The egocentric view.* The camera sees the wearer's hand from behind and often only in part. The target may be touched or across the room, and a pointing direction drawn in the image is only a rough guide to it.
- *Other people's hands.* A companion's hand or a stranger's pointing gesture can be in view, and must not be mistaken for the wearer's.
- *The budget.* Glasses-class processors share a battery of a few hundred milliampere-hours with the camera, audio and radio. Sending frames to a cloud model adds network latency and moves imagery of bystanders off the device.

Prior work. Pointing-target grounding in egocentric images is an active research area, and EgoPoint-Ground [20] provides a large benchmark for it. The strongest methods are large models:

- multimodal LLMs that reason step by step from the hand to a ray and then to the target [20, 21];
- a 226M-parameter transformer that models the pointing ray explicitly [19].

Some benchmark tasks even give the model the ground-truth hand box [20]. These systems reach high accuracy on a GPU, but none of them is designed for glasses-class hardware, and to our knowledge none addresses other people's hands in view. Egocentric fingertip systems, including one that runs on a phone, stop at the fingertip [14, 15] and never localise the target.

SpatialNet. We ask how far a model that fits on the glasses can go. SpatialNet needs only one 2D RGB frame:

- no text prompt, no hand box, no depth;
- no external hand detector or pose estimator.

It predicts the wearer's hand, the fingertip, the pointing direction and the box of the pointed-at object, with presence gates for each. All outputs come from one forward pass of a compact convolutional network with no self-attention and no token-by-token decoding: 0.72M parameters,

0.23 GFLOPs, 1.6 MB in FP16. This makes it 300 times smaller than VistaRef and about 10,000 times smaller than the 7B multimodal LLMs.

One design choice lets such a small network localise precisely: *instance-anchored heads*. The heads form a chain in which each one reads a shared feature map at the location the previous one predicted, so the fingertip, direction and target all belong to the same hand.

We also give the target head the geometry of the predicted ray, as six ray-coordinate maps entering through a zero-initialised branch. We then test whether a model this small makes use of it, rather than assuming so. Training additionally pastes unlabelled hands of other people into the frames and penalises responses to them, so the model learns to prefer the wearer's hand without labelled bystander hands.

Evaluation. We train and test on EgoPoint and report:

- accuracy by fingertip–target distance, which matters because about 70% of targets are touched by the fingertip;
- comparisons with published results of state-of-the-art models and zero-shot multimodal LLMs on the same test split, and with simple fingertip-based references;
- ablations of each component;
- behaviour when other people's hands are in view (pasted and real);
- latency, memory and battery drain on an engineering unit of a pair of smart glasses.

Results in brief.

- SpatialNet finds the wearer's hand reliably (P@0.5 of 92.6%, the share of test frames with IoU≥0.5) and the fingertip precisely (3.9 px mean error at 160×160).
- It localises the pointed-at object with a P@0.5 of 58.9% (mIoU 53.3%). That is comparable to zero-shot 7–8B multimodal LLMs reported on the same test split (44.5–49.8%), close to a 7B model fine-tuned on EgoPoint (63.6%), and far below VistaRef (93.9%) and PointVG-R (83.6%).
- It drops to 25.4% for far targets. Trained at higher input resolution, with no new parameters, the same model reaches 62.4% at 224 px (far: 35.9%) and 65.1% at 320 px (Section 7.3).
- It runs in 60–80 ms per frame on the glasses. Running it continuously costs less battery (9% per minute on the test unit) than streaming frames to a phone at 0.5 fps (16%).

We present this as an operating point, with its limits measured, not as a replacement for large models.

Contributions.

(1) *An image-only, single-pass pointed-target localiser that runs on smart glasses.*
   - From one RGB frame, a 0.72M-parameter CNN without self-attention, external models or prompts outputs the hand, fingertip, direction and target box.
   - On a smart-glasses engineering unit it takes 60–80 ms per frame, and running it continuously costs less battery than streaming frames off the device (Sections 4, 6.2 and 7.6).

(2) *Instance-anchored heads that keep a sub-million-parameter model precise.* Against a plain model of the same size, the anchoring improves:
   - P@0.7, by 2.8 points;
   - the hand box, by 1.8 points of mIoU;
   - the direction, by 3.4°;
   - small targets, by 11.3 points.

   Each gain exceeds the variation between training seeds (Sections 4.3 and 7.4).

(3) *An analysis of what limits pointing localisation at this scale.*

- Input resolution is the largest lever, at no parameter cost.
- Distractor pasting makes the model robust to other hands at no cost on clean images.
- A prior learned from explicit ray geometry collapses into a fingertip-proximity prior when most targets are touched. This explains why far targets stay hard, and indicates how future models should constrain the ray (Sections 7.3–7.5).

(4) *An evaluation that places accuracy next to cost.* It compares with published models that are 300 to 10,000 times larger, stratifies by fingertip–target distance, and decomposes the errors (Section 7).

Distractor pasting is treated as a supporting result rather than a headline contribution. Its effect is clear (Section 7.4), but its test distractors are synthetic.

2 Related Work

Pointing as input. Deictic gestures have been part of multimodal interfaces since *Put-that-there* [4]. Studies of mid-air pointing show that both the hand and the eye matter. Ray-casting models of pointing have systematic offsets [27], and taking the dominant eye and handedness into account makes them markedly more accurate [30]. On wearables, GazePointAR combines gaze, pointing and conversation history to resolve spoken pronouns in AR [17]. EgoPointVQA asks questions about pointed-at objects in egocentric video and feeds 3D hand keypoints, estimated by an off-the-shelf model, to a multimodal LLM as hand-intent tokens [6]. These systems show the value of pointing on wearables; SpatialNet supplies the localisation step that such systems need, at a cost that fits on the glasses.

Fingertips and pointing directions. EgoFinger detects the hand and fingertip in egocentric video for air-writing [14]. GestARLite runs a hand detector, a fingertip regressor and a gesture classifier on a smartphone [15]. DeePoint recognises pointing and estimates its 3D direction from third-person video of the whole body [28]. None of these localises the object being pointed at.

Pointing-target grounding.

- *Third-person.* YouRefIt grounds targets from language and gesture together [5].
- *EgoPoint-Ground.* For egocentric images, it introduced 15K images (real captures from smart glasses, plus synthetically composed and generated images) and several tasks [20]. In its pointing-only task, models receive the image and the ground-truth hand box and are evaluated without task-specific training. Its SV-CoT baseline reasons with a multimodal LLM from the pointing direction to a ray and then to the target.
- *PointVG-R.* It builds a similar hand–direction–ray–target chain of thought into Qwen2.5-VL-7B with reinforcement learning [21], and names latency as a limitation for wearables.
- *VistaRef.* It adds hand-keypoint and geometric-ray modules and an orientation-consistency loss to a BEiT-3 grounding model (226M parameters with the Base backbone) [19]. With the Large backbone, it reports a P@0.5 of 93.9% (mIoU 82.0%) on the 3,072 EgoPoint test images (Section 5 explains how our test split differs).
- *EgoPoint-Bench.* It shows that general multimodal LLMs still struggle with egocentric pointing [18].

Explicit ray reasoning is thus not new, and neither is avoiding a hand-box input: VistaRef uses a fixed prompt and finds the hand itself. What SpatialNet changes is the operating point:

- the only input is the image;
- it uses no language model, no transformer and no external pose model;
- a single forward pass of a 0.72M-parameter CNN, 300–10,000 times smaller than the models above, produces every output;

Table 1. Methods that localise pointing or its target. *Hand box*: the ground-truth hand box is an input. Parameter counts as reported; ">1B": built on a multimodal LLM.

Method	View	Test-time input	Output	Params	On device	Other hands
EgoFinger [14]	ego	image	fingertip	–	–	–
GestARLite [15]	ego	frames	fingertip, gesture	16.3 MB	phone	–
DeePoint [28]	3rd	video	pointing, 3D direction	–	–	–
YouRefIt [5]	3rd	image + language	target box	–	–	–
SV-CoT (POG) [20]	ego	image + hand box	target box	>1B	–	–
PointVG-R [21]	ego	image + instruction	target box	7B	–	–
VistaRef [19]	ego	image + fixed prompt	target box, keypoints	≥226M	–	–
SpatialNet	ego	image	hand, tip, ray, target	0.72M	glasses	trained

- we test, rather than assume, whether explicit ray geometry helps a model this small, and show when it does not;
- it runs on smart-glasses hardware.

Table 1 summarises the differences.

Hands of the wearer and of others. Egocentric hand datasets have long distinguished the camera wearer's hands from other people's: EgoHands labels *my* and *your* left and right hands [2]. Large-scale hand detectors model hand side and contact state [31]. Pointing datasets, EgoPoint included, contain only the wearer's hand. We therefore synthesise other people's hands with copy-paste augmentation [8, 12]. Unlike in standard copy-paste, the pasted instances are deliberately left unlabelled and penalised.

Efficient localisation and grounding. Our trunk follows efficient detection design [26, 34]. Our heads predict keypoint heatmaps in the manner of CenterNet [41], with a differentiable soft-argmax [33]. Open-vocabulary detectors and multimodal LLMs can ground text queries [1, 24, 29, 38], but pointing is not text, and their size rules them out for continuous use on glasses.

3 Problem Definition

Outputs. Given one RGB frame from a head-worn camera, SpatialNet predicts:

(1) three presence flags (H, P, T): the wearer's hand is visible, the hand is pointing, and something is pointed at;
(2) the box $\mathbf{b}_h$ of the wearer's hand;
(3) the fingertip $\mathbf{p}$;
(4) the pointing direction $\mathbf{u}$ in the image plane;
(5) the box $\mathbf{b}_t$ of the pointed-at object, which we call the *target*.

Boxes are (c_x, c_y, w, h) and points (x, y), both normalised to $[0, 1]$. The flags combine into four scenes that an interface can act on: 000 *no hand*, 100 *hand, not pointing*, 110 *pointing, no target* and 111 *pointing at a target*.

What separates this from generic detection.

- *Ownership.* When several hands are visible, $\mathbf{b}_h$, $\mathbf{p}$ and $\mathbf{b}_t$ must refer to the wearer's hand, and a frame showing only other people's hands is *no hand*.
- *Budget.* The model must run on the glasses' own processor, within an interactive latency, without a second network or a server call.

Scope. We do not recognise *what* the target is. $\mathbf{b}_t$ is the region that a later step would take as input: an object recogniser, a visual question-answering model or an application. Localising the region first means that this step can run on a small crop instead of the full frame, and only when a pointing event occurs. The cost of that later step is outside the scope of this paper.

4 SpatialNet

Figure 2 shows the complete network. A shared trunk turns the frame into one feature map. Four heads then run in a fixed order (wearer's hand, fingertip, pointing direction, pointed target), and each head reads the shared map *at the location that the previous head predicted*. We call these heads *instance-anchored*: the fingertip is searched for around the hand that was found, the direction is computed from features of that hand and fingertip, and the target is searched for along the resulting ray. Three small gates decide which outputs are valid. Everything is a single forward pass; there is no external hand detector, no language input, and no hand box at test time.

4.1 Shared Trunk

The frame is resized to 160×160. A ShuffleNetV2 0.5× backbone [26] (stages 2–4 only; ImageNet-pretrained [7]) gives feature maps at strides 8, 16 and 32. Two BiFPN layers [34] merge them into one $192 \times 20 \times 20$ map at stride 8. Each fused map is smoothed by a depthwise 3×3 convolution, SiLU, and parameter-free SimAM attention [37]. We then append two coordinate channels (CoordConv [23]) and reduce to D=256 channels with a depthwise-separable convolution. Two ConvMixer-style residual blocks [35] follow, each made of a depthwise 5×5 and a pointwise convolution with residual connections. Their batch-norm scales start at zero, so each block is the identity at initialisation. The output is the shared map $S \in \mathbb{R}^{256 \times 20 \times 20}$, which all heads read.

A 20×20 grid gives 8-pixel cells at the model input. The heads recover points with sub-cell precision (Section 4.2), so a finer grid would mostly add compute. Sharing one map keeps the heads cheap: the largest head has 104K parameters (Table 2).

4.2 Locating One Instance: Peak-Windowed Soft-Argmax

Each head predicts a 20×20 logit map H and turns it into a point. A global soft-argmax [33] returns the probability-weighted mean over all cells. When two hands are visible, that mean falls *between* them. We therefore restrict the softmax to a $(2r{+}1) \times (2r{+}1)$ window $\mathcal{W}$ around the arg-max cell $k^\star$:

$$\pi_k = \frac{\mathbb{1}[k \in \mathcal{W}]\ \exp(\tau H_k)}{\sum_{j \in \mathcal{W}} \exp(\tau H_j)}, \qquad \hat{\mathbf{c}} = \sum_k \pi_k\, \mathbf{g}_k, \tag{1}$$

where $\mathbf{g}_k \in [0,1]^2$ is the normalised position of cell k and τ is a temperature. The estimate follows a single peak, is differentiable within the window, and is not limited to cell centres. We use $(\tau, r) = (8, 2)$ for the hand and the target and $(10, 1)$ for the fingertip. The peak logit $H_{k^\star}$ is also passed to the gates as a confidence cue.

4.3 Instance-Anchored Heads

Wearer's hand. A depthwise 3×3 and pointwise head gives hand-centre logits H^h, and Eq. (1) gives the centre $\mathbf{c}_h$. For the box size (w_h, h_h), we sample a 3×3 neighbourhood of S around $\mathbf{c}_h$ bilinearly (one-cell spacing). A depthwise 3×3 convolution and a pointwise projection collapse it to 128 dimensions, and a two-layer MLP with a sigmoid maps it to the size, giving $\mathbf{b}_h = (\mathbf{c}_h, w_h, h_h)$. To describe *this* hand instance, we sample a 5×5 RoIAlign-style lattice [13] over $\mathbf{b}_h$ enlarged by 1.15. This lattice is collapsed to 128 dimensions and fused with the box coordinates into the hand descriptor $\mathbf{v}_h \in \mathbb{R}^{128}$.

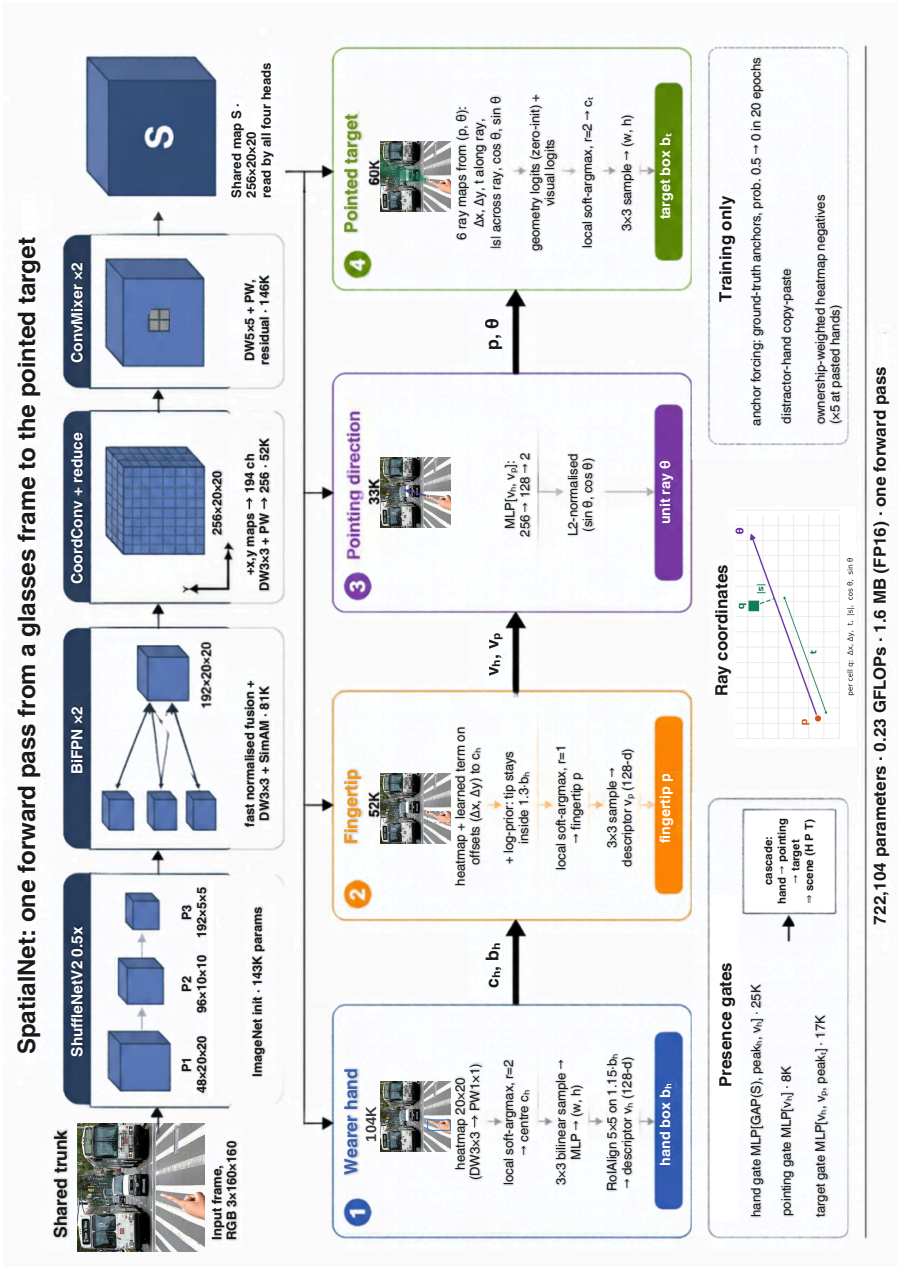


Fig. 2. SpatialNet. The trunk (top) builds one shared map S. Four heads (middle) run in order, each reading S where the previous one predicted: hand centre and box $\mathbf{c}_h, \mathbf{b}_h$; descriptors $\mathbf{v}_h, \mathbf{v}_p$; fingertip and direction $\mathbf{p}, \theta$. Bottom: presence gates, the ray coordinates of Eq. 4, and training-only mechanisms. Head panels give parameter counts; thumbnails show ground-truth labels of one test image.

Fingertip. The fingertip must belong to the hand just found. Its logits add three terms:

$$H^p = V^p(S) + G^p\big([S;\, \alpha_p(\mathbf{g} - \mathbf{c}_h)]\big) + B(\mathbf{b}_h), \tag{2}$$

- V^p is a visual head of the same form as the hand head.
- $\alpha_p(\mathbf{g} - \mathbf{c}_h)$ are two channels holding each cell's offset to the hand centre; α_p is a learnable scale.
- G^p is a small branch (a 1×1 convolution to 48 channels, then a 3×3 convolution to one channel) that learns where fingertips lie relative to the hand centre. Its last layer starts at zero.
- B is a fixed log-prior. It is zero inside the hand box dilated by δ=1.3 and falls linearly outside it:

$$B_k(\mathbf{b}_h) = -\frac{\lambda}{\tau_p}\left(\left[\frac{|g_{k,x} - c_{h,x}|}{\delta w_h/2} - 1\right]_+ + \left[\frac{|g_{k,y} - c_{h,y}|}{\delta h_h/2} - 1\right]_+\right), \tag{3}$$

  where $[\cdot]_+ = \max(\cdot, 0)$. With λ=4, the penalty after the softmax temperature τ_p is 4 logits for every half box-width outside the dilated box. No gradient flows from the prior into $\mathbf{b}_h$.

In the training labels, the fingertip lies inside the dilated box in 99.4% of pointing images, so the prior rarely contradicts a label, while it rules out the fingertips of other hands. Eq. (1) (with r=1) gives the fingertip $\mathbf{p}$, and a 3×3 sample around it gives the fingertip descriptor $\mathbf{v}_p \in \mathbb{R}^{128}$.

Pointing direction. An MLP ($256 \to 128 \to 2$, tanh) maps $[\mathbf{v}_h; \mathbf{v}_p]$ to a vector. We L2-normalise it into the unit direction $\mathbf{u} = (\cos\theta, \sin\theta)$ in image coordinates. The direction is supervised with the image-plane direction from the labelled fingertip to the labelled target centre (Section 4.5). $\mathbf{u}$ is therefore the direction *to the target* as it appears in the image, not the 3D orientation of the finger.

Pointed target. The target head is given the geometry of the predicted ray explicitly. From $\mathbf{p}$ and $\mathbf{u}$, every cell k receives six *ray coordinates* (inset in Figure 2):

$$\Delta_k = \alpha_t(\mathbf{g}_k - \mathbf{p}), \qquad t_k = \Delta_k \cdot \mathbf{u}, \qquad s_k = \big|\Delta_{k,x} \sin\theta - \Delta_{k,y} \cos\theta\big|, \qquad \cos\theta, \qquad \sin\theta. \tag{4}$$

These are:

- Δ_k (two channels): the scaled offset from the fingertip;
- t_k: the signed distance along the ray, negative behind the fingertip;
- s_k: the distance from the line of the ray;
- $\cos\theta$ and $\sin\theta$: the direction itself.

The target logits are

$$H^t = V^t(S) + G^t\big([S;\, R(\mathbf{p}, \theta)]\big), \tag{5}$$

where R stacks the six maps, and G^t (13K parameters) has the form of G^p and also starts at zero. The soft-argmax applies a softmax to τH^t, so $\pi_k \propto \exp(\tau V^t_k)\exp(\tau G^t_k)$: the geometry term acts as a learned *multiplicative prior* on the visual evidence. Because G^t starts at zero, training begins with a purely visual head, and the network uses the ray only as far as it lowers the loss.

We let the network learn the shape of the prior, instead of fixing a cone, because the right shape depends on the situation. When the fingertip is on the object (about 70% of EgoPoint targets, Section 5), the target centre can lie in any direction from the fingertip, even slightly behind it. For distant targets, a narrow band along the ray is informative. Whether the network exploits this freedom is an empirical question; Section 7.4 shows that on EgoPoint it learns a fingertip-proximity prior instead. The target centre $\mathbf{c}_t$ follows from Eq. (1). Its size comes from a 3×3 sample, as for the hand, giving $\mathbf{b}_t = (\mathbf{c}_t, w_t, h_t)$.

Table 2. Parameters per module. FLOPs are counted by PyTorch's `FlopCounterMode` (one multiply–accumulate = 2 FLOPs; bilinear sampling not counted) for one 160 × 160 frame. Sizes are serialised state dictionaries.

Module	Parameters	Role
ShuffleNetV2 0.5×, stages 2–4	143,136	backbone, ImageNet-pretrained
BiFPN ×2	81,426	multi-scale fusion to stride 8
CoordConv + reduce	52,310	194 → 256 channels
ConvMixer ×2	145,920	context mixing (5 × 5 depthwise)
Hand head	104,259	heatmap, size, descriptor $\mathbf{v}_h$
Fingertip head	51,826	incl. 12,913 in the hand-offset branch G^p
Direction MLP	33,154	$[\mathbf{v}_h; \mathbf{v}_p] \to \mathbf{u}$
Target head	60,404	incl. 13,105 in the ray branch G^t
Gates (hand / pointing / target)	24,769 / 8,321 / 16,577	presence logits
Learnable scales α_p, α_t	2	
Total	**722,104**	0.231 GFLOPs; 3.0 MB FP32; 1.6 MB FP16

4.4 Presence Gates and Scene Decoding

Three MLP gates output logits for:

- *the wearer's hand is present*, from $[\mathrm{GAP}(S);\ H^h_{k^\star};\ \mathbf{v}_h]$;
- *the hand is pointing*, from $\mathbf{v}_h$ alone;
- *there is a pointed target*, from $[\mathbf{v}_h; \mathbf{v}_p; H^t_{k^\star}]$.

At inference, they are read as a cascade with threshold 0.5:

(1) If the hand gate is off, the frame is *no hand.*
(2) Otherwise, if the pointing gate is off, it is *hand, not pointing.*
(3) Otherwise, if the target gate is off, it is *pointing, no target.*
(4) Otherwise, it is *pointing at a target.*

The pointing and target gates read instance descriptors, not the whole image, so another person's pointing hand elsewhere in the frame cannot switch them on by itself. The hand gate also sees the global pool, so that a frame containing only other people's hands can be classed as having *no wearer's hand.* EgoPoint contains only the first and last of the four scenes (Section 5). We therefore evaluate the hand gate and make no claims about the pointing and target gates.

4.5 Training Objective

The loss is a weighted sum of 13 terms (weights in Appendix A). Each term is computed only on images whose label contains the parts it needs.

Supervised localisation.

- Hand box $\mathbf{b}_h$: CIoU loss [40].
- Fingertip $\mathbf{p}$: Wing loss [10], on normalised coordinates with w=0.5 and ϵ=0.1.
- Direction: $1 - \cos(\mathbf{u}, \mathbf{u}^\star)$, where $\mathbf{u}^\star$ is the unit vector from the labelled fingertip to the labelled target centre.
- Target box $\mathbf{b}_t$: SIoU loss [11] plus 1 − IoU.

Heatmaps. The penalty-reduced focal loss of CornerNet and CenterNet [16, 41] (α=2, β=4), on Gaussian targets at the hand centre, fingertip and target centre (σ = 1.5, 0.8 and 1.2 cells). These give each heatmap a direct signal, in addition to the gradient that reaches it through the soft-argmax.

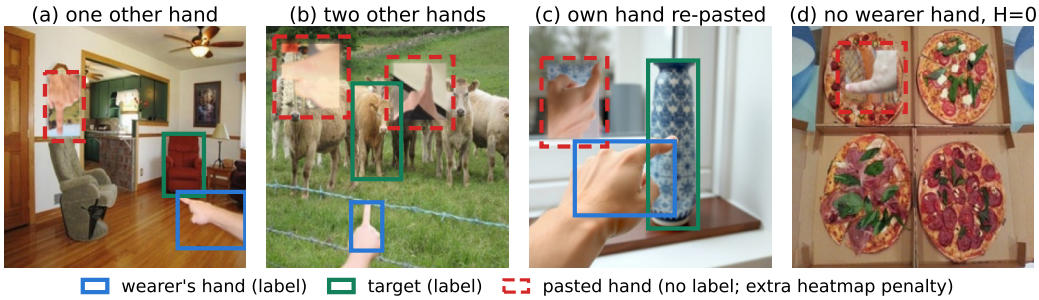


Fig. 3. Distractor-hand pasting, produced by the training code. Pasted hands (red, dashed) come from other images (a, b) or from the same image (c), are rotated so that they rarely enter from the bottom edge, and are never labelled. A no-hand image with a pasted hand keeps the label *no hand* (d).

Gates. Focal binary cross-entropy [22] (γ=2), each gate on its own population: the hand gate on all images, the pointing gate on hand images, and the target gate on pointing images. Positives are weighted 2× (3× for the hand gate), because a missed pointing gesture costs the user more than a false alarm that the interface can ignore.

Geometric consistency. Two terms use predictions only.

The *ray-alignment* term asks the predicted target centre to lie on the predicted ray, in front of the fingertip:

$$\mathcal{L}_{\text{ray}} = \left\| \mathbf{r} - (\mathbf{r} \cdot \hat{\mathbf{u}})\, \hat{\mathbf{u}} \right\| + 0.5 \left[-\mathbf{r} \cdot \hat{\mathbf{u}} \right]_+, \qquad \mathbf{r} = \hat{\mathbf{c}}_t - \hat{\mathbf{p}}. \tag{6}$$

- $\mathbf{u}^\star$ runs from the labelled fingertip to the labelled target centre, so the ground truth gives $\mathcal{L}_{\text{ray}} = 0$ exactly. The term therefore does not move the optimum; it only penalises predictions that disagree with each other.
- For a fixed angular error, the term grows with the fingertip–target distance, which puts more weight on far targets.

The *hand-axis* term is a hinge that keeps $\hat{\mathbf{u}}$ within 60° of the axis from the predicted hand centre to the predicted fingertip; it is zero inside this cone.

- It is only a weak plausibility prior. The labelled direction lies inside the cone in 68.6% of training images: 64.1% when the fingertip is on the object and 83.6% for far targets.
- For that reason it has the smallest weight of all 13 terms. Ablation A2 (Section 7.4) removes both consistency terms.

4.6 Learning Whose Hand It Is

No public pointing dataset labels other people's hands, and EgoPoint frames contain only the wearer's. We therefore create distractors during training (Figure 3). The approach follows copy-paste augmentation [8, 12], but with the opposite aim: pasted hands are *not* labelled, and the model is penalised for responding to them.

Pasting.

- *Frequency.* With probability 0.5 for images with a hand and 0.3 for images without one, we paste one or two hand crops taken from other training images.
- *Size.* Each crop is scaled to 0.45–1.0 of its source size.
- *Orientation.* With probability 0.7, the crop is rotated by 90, 180 or 270° and possibly mirrored. Unlike the wearer's arm, it therefore rarely enters from the bottom edge.

- *Placement.* The crop is centred in the upper 65% of the frame, with less than 15% overlap with the wearer's hand and the target.
- *Blending.* The crop is blended with a feathered alpha mask and a small colour jitter.
- *Self-paste.* In 30% of cases, the pasted crop is the wearer's own hand from the same image. Appearance alone then cannot tell the two hands apart, and the model has to use position, orientation and context.
- *Decoys.* With probability 0.15, we also paste an unlabelled patch that contains no hand, so that a pasted rectangle is not itself a cue.

Labels never change. In particular, a no-hand image with pasted hands remains *no hand*, which teaches the hand gate to answer for the wearer only.

Ownership-weighted negatives. In the hand-heatmap loss, the negative term near a pasted hand is multiplied by $1 + 4\,G_k$, where G_k is a Gaussian (σ=1.5 cells) at that hand's centre. A response on another person's hand thus costs up to five times as much as one on background.

The prior this encodes. The procedure assumes that, from a head-worn camera, the wearer's arm enters from the bottom of the frame, while other people's hands appear higher and in other orientations. This holds for most use of glasses but not always, for example when the wearer raises a hand above eye level or when a companion's hand is near the bottom of the view. Section 8 discusses this limitation.

4.7 Scheduled Anchor Forcing

Early in training, the predicted hand centre, fingertip and target centre are close to random, so the downstream heads would read features at meaningless locations. As in scheduled sampling for sequence models [3], we sometimes replace each anchor with its ground-truth location. This happens independently per image and per anchor, with a probability that decreases linearly from 0.5 to 0 over the first 20 epochs. From then on, training uses the model's own anchors, as at test time.

5 Data and Evaluation Protocol

5.1 EgoPoint

We train and test on the image set of EgoPoint-Ground [20], which we call EgoPoint. Its paper reports 15,338 images:

- 39% (6,035) real captures with RayNeo V3 smart glasses (12 MP Sony IMX681 sensor);
- 39% (5,976) synthetically composed;
- 22% (3,327) LLM-generated.

These are the paper's main-text labels; its appendix calls the last two groups "synthetic generation" and "image editing". The paper describes a random 7:2:1 train:val:test split, plus a domain-adaptive split that keeps all real captures for testing. The release we use has 15,455 images split 10,837 / 1,546 / 3,072. VistaRef [19] and PointVG-R [21] report the same counts, so our test images are theirs, minus the exclusions below. We use this released split as is, so that our numbers are directly comparable with theirs. Each positive record shows one wearer's hand pointing at one target; negatives show no hand. The release carries no per-image source label (real, composed or generated), so, like VistaRef and PointVG-R, we report on the whole test split.

Conversion. We convert each record to our label format (the three flags, hand box, fingertip and target box):

- Negatives become 000, and their object boxes are ignored.
- Positives with a hand box, an annotated fingertip and a target box become 111.

Table 3. EgoPoint after conversion. *On object*: the fingertip lies inside the target box; *near*: fingertip–box distance ≤ 0.08 of the image side; *far*: > 0.08.

				Pointing at target, by distance		
Split	Images	No hand (000)	Pointing at target (111)	On object	Near	Far
Train	10,669	1,535	9,134	6,511	1,995	628
Validation	1,524	211	1,313	922	297	94
Test	3,029	425	2,604	1,796	599	209
Total	15,222	2,171	13,051	9,229	2,891	931

- In three test records, the hand label was attached to an object and the hand was labelled as an object. We detected these by the fingertip position and swapped the labels back.
- We dropped 232 positives: 229 without an annotated fingertip (164 train, 22 validation, 43 test) and 3 without a hand box. All 229 are flagged as unclear in the source annotations. Spot checks showed open palms, held objects, or a hand box on the hand that was not pointing.

Table 3 summarises the result. Images are stored at 224×224; the model squashes every frame to its square 160×160 input, regardless of the original aspect ratio.

Consequences for evaluation.

- *Test split.* Our test split has 3,029 images, 43 positives fewer than the official one. Wherever we show numbers published on the official split, we say so; those numbers are not strictly comparable (Section 7.1).
- *Two scenes only.* EgoPoint contains only scenes 000 and 111. The pointing and target gates therefore never see a negative example, and we report neither their accuracy nor four-class scene accuracy.
- *Distance skew.* About 70% of targets are touched by the fingertip, and only 7% are far. Pooled results are dominated by touched targets, so we report every target metric by distance as well. The median distance from the fingertip to the target centre is 0.12, 0.17 and 0.32 of the image side for on-object, near and far targets (test split).

5.2 Metrics

- *Target and hand localisation.* We use the two standard grounding metrics [19–21]. *mIoU* is the IoU between predicted and labelled box, averaged over images, with a 95% bootstrap confidence interval (1,000 resamples [9]). *P@τ* (Precision@τ) is the share of images with IoU$\geq\tau$, for τ = 0.3, 0.5 and 0.7. Both are given in %. Target metrics are given overall and by distance.
- *Fingertip and direction.* Fingertip error is in pixels at the 160×160 input (1 px = 0.63% of the image side). Direction error is the angle between $\hat{\mathbf{u}}$ and $\mathbf{u}^\star$.
- *Hand presence.* For the hand gate, we report accuracy, missed hands, false alarms and AUROC, and the two-scene accuracy (no hand versus pointing at target), with Wilson intervals [36] for proportions.
- *No skipped images.* Localisation is scored on every image whose label contains the part, whatever scene the model predicted, so hard images cannot be skipped.

Ownership test. To test robustness to other people's hands, we paste one or two hands into every test image. The hands come from other test images, 30% are re-pasted copies of the wearer's own hand, and their positions are fixed by a seed. Labels stay unchanged. We report:

- *ownership accuracy*: the share of hand images in which the predicted hand centre is nearer the wearer's hand than any pasted hand;
- how often the predicted centre falls inside a pasted hand;
- the share of no-hand images that stay *no hand*.

These distractors follow the training distribution, so this test measures robustness to that synthetic distribution and may overstate robustness in real scenes. Section 7.5 therefore adds real frames.

Model selection. The reported checkpoint has the lowest validation loss. Training, early stopping and checkpoint selection use only the training and validation splits; the test split is evaluated once, after training. The ablation, resolution and seed runs follow the same rule and do not change the reported model.

6 Implementation and On-Glasses Deployment

6.1 Training

Optimisation.

- AdamW [25], weight decay 10^{-4}, batch size 32.
- One-cycle schedule [32]: peak learning rate 3×10^{-3} for the heads and 3×10^{-4} for the backbone, 10% warm-up, then cosine decay.
- Up to 180 epochs, stopping early after 25 epochs without a lower validation loss.
- Gradient clipping at 1.0, mixed precision, seed 42.

Training took 1 h 40 min on one NVIDIA Tesla T4 (47 s per epoch). Early stopping ended it after 125 epochs; the reported checkpoint is from epoch 100. Early stopping (patience 25 on the validation loss) ended the 180-epoch one-cycle schedule before it completed. Every run in this paper uses the same rule, so all comparisons are like for like; a completed schedule would, if anything, raise the absolute numbers.

Augmentation. Besides the distractor pasting of Section 4.6:

- horizontal flip (no vertical flip, because a wearer's hand entering from the top would contradict the ownership prior);
- shift, scale and rotation (±5%, ±10%, ±15°; probability 0.4);
- perspective warp (0.2);
- brightness and contrast (±20%; 0.5);
- Gaussian blur (0.2);
- mixup [39] between images of the same scene, with $\lambda \sim \mathrm{Beta}(0.2, 0.2)$ folded to ≥ 0.5 and the label of the dominant image.

All augmentation is on the fly; the dataset on disk is never modified. Appendix B lists all parameters.

Export. The model is exported to ONNX (opset 17, which bilinear grid sampling requires) and to FP16. The FP16 weights take 1.6 MB.

6.2 Deployment on Smart Glasses

We deploy the 160 px model on an engineering unit of a pair of smart glasses: a pre-production test device, not a retail product. Its hardware:

- a smart-glasses system-on-chip paired with a low-power microcontroller;
- a single 12 MP ultra-wide camera (105°) in portrait orientation, with a dedicated low-power vision NPU;
- Wi-Fi 6 and Bluetooth 5.4 to a companion phone;
- a 160 mAh battery, in a frame weighing under 45 g.

Because it is an engineering device, its power management is not tuned for production, so absolute battery figures are pessimistic. We therefore compare power between operating modes on the same unit, rather than reporting battery life. We treat the unit as a measurement platform: its pre-production software stack is not public, so we report end-to-end device measurements rather than runtime-specific details.

Operating modes. We compare two ways of running pointing localisation:

- *On the glasses:* SpatialNet runs continuously on frames from the glasses' camera.
- *Offloading:* the glasses only capture frames and transmit them to the companion phone at 0.5 frames per second, the alternative in which localisation would run on the phone or in the cloud.

Measurement.

- the inference latency per frame;
- the peak additional RAM used during inference, measured over the idle application with the model already loaded (the 1.6 MB of FP16 weights are stored separately);
- the battery drain over one minute, in each operating mode, starting from the same charge level.

The camera delivers portrait frames, whereas most EgoPoint images are landscape. The model squashes both to a square, so the deployed input has a different aspect ratio from most training images. Real-frame behaviour is assessed in Section 7.5.

7 Results

We answer five questions:

(1) How well does SpatialNet find the pointed-at object, compared with published results of state-of-the-art and zero-shot models and with simple references (Section 7.1)?
(2) Where does it fail, by distance and by head (Section 7.2)?
(3) Which design choices matter (Section 7.4)?
(4) Does it follow the wearer's hand when other hands are visible (Section 7.5)?
(5) How fast does it run on the glasses (Section 7.6)?

SpatialNet numbers are from the seed-42 run on the 3,029 test images; this is the model deployed on the glasses. We retrained it with two more seeds (1 and 2) to measure how much retraining alone changes the results:

- The headline is stable: P@0.5 is 58.9–59.0% in all three runs, and P@0.7 and mIoU vary by under 0.8 points.
- Far targets (25.4–27.8%) and the median direction error (18.6–20.2°) vary more.
- Small targets vary most (27.8–36.9%), because only 309 test images have one; seed 42 is the highest of the three.

We use this spread to calibrate the single-run comparisons below (Sections 7.3 and 7.4).

7.1 Pointed-Target Localisation

References. We compare SpatialNet with two kinds of reference.

- *Published results.* We copy these numbers and do not re-run the models.
  - VistaRef [19] and PointVG-R [21] report results on the same 3,072-image test split; ours omits 43 unclear images and the 425 images without a target. PointVG-R also reports zero-shot and fine-tuned multimodal LLMs on that split.

Table 4. Localisation of the pointed-at target on the EgoPoint test split. All columns compare a predicted *target* box with the labelled target box; hand accuracy is in Table 5. Our rows use the 2,604 test images with a target; the 425 no-hand images have none. mIoU and P@τ (Section 5.2) in %, the published values multiplied by 100. We count IoU$\geq\tau$, as does VistaRef; EgoPoint-Ground and PointVG-R count IoU$>\tau$. *Far*: P@0.5 on the 209 far targets. Latency per image on the hardware named in the text; only SpatialNet runs on the glasses.
* VistaRef [19], Table 1, setting "Train-Real Data Test", BEiT-3-L backbone (226M parameters with BEiT-3-Base); test set of 3,072 images, which includes no-hand images.
‡ PointVG-R [21], Table 1, EgoPoint-CoT test set, built on the same 15,455-image release and 3,072-image test split.
† EgoPoint-Ground [20], Table 4, pointing-only (POG) task: inference only, given the ground-truth hand box; the evaluated data subset is not stated.
How the published works score images without a target is not stated; our rows exclude them.

Method	Params	P@0.3	P@0.5	P@0.7	mIoU	Far	Latency
Published, trained on EgoPoint (not re-run)							
VistaRef*, image + fixed prompt	>226M	97.0	93.9	82.6	82.0	–	–
PointVG-R‡, image + instruction, SFT + RL	7B	88.1	83.6	74.5	75.7	–	–
Qwen2.5-VL-7B‡, image + instruction, SFT	7B	68.7	63.6	56.7	59.8	–	–
Published, zero-shot (not re-run)							
Qwen2.5-VL-7B‡, image + instruction	7B	54.2	49.8	42.9	45.5	–	–
Qwen3-VL-8B‡, image + instruction	8B	48.1	44.5	39.0	43.0	–	–
SV-CoT†, image + hand box	>1B	69.4	63.9	55.6	–	–	–
Qwen3-VL-4B†, image + hand box	4B	67.1	61.2	53.2	–	–	–
Simple references							
Mean training box, fixed position	0	41.3	9.6	0.9	27.4	4.3	–
Mean-size box at predicted fingertip	0.72M	43.5	11.2	0.9	28.8	0.0	–
Mean-size box at true fingertip (oracle)	–	43.6	11.4	1.0	29.0	0.0	–
SpatialNet (image only, seed 42)	0.72M	78.6	58.9	31.8	53.3	25.4	60–80 ms
mean ± std, 3 seeds		78.1±0.6	59.0±0.1	31.5±0.4	52.9±0.5	26.3±1.3	
SpatialNet, 224 px input (same model)	0.72M	78.8	62.4	37.1	54.8	35.9	–
SpatialNet, 320 px input (same model)	0.72M	79.4	65.1	40.3	56.5	34.9	–

  - EgoPoint-Ground's pointing-only (POG) results [20] come from a different setting: zero-shot models receive the ground-truth hand box, and the evaluated subset is not stated.
- *Simple references.* These test whether knowing where the fingertip is would be enough:
  - the mean training target box at a fixed position;
  - a box of mean training size centred on SpatialNet's predicted fingertip;
  - the same box centred on the ground-truth fingertip.

Results. Table 4 gives three findings.

- *The target head does real work.* Knowing the fingertip is not enough: a mean-size box at the true fingertip reaches a P@0.5 of only 11.4% (12.0% when shifted by the mean training offset), because target sizes vary widely (2.6% to 44% of the image area, 10th to 90th percentile). SpatialNet reaches 58.9%.
- *Large models trained on EgoPoint are far more accurate.* SpatialNet is 35 points below VistaRef in P@0.5 and 51 points below in P@0.7, where its boxes are least precise; in mIoU the gap is 29 points (53.3 against 82.0). It is 25 points below PointVG-R, and 4.7 points below Qwen2.5-VL-7B fine-tuned on the same data.
- *It is comparable to zero-shot multimodal LLMs.*

Table 5. Accuracy of each head by fingertip–target distance (test split, pointing-at-target images). Fingertip error in pixels at the 160 × 160 input; direction error in degrees. mIoU and P@0.5 in %; brackets: 95% bootstrap confidence intervals of the mIoU.

	Hand		Fingertip (px)		Direction (°)		Target	
Distance (images)	mIoU	P@0.5	mean	<5 px	mean	<15°	mIoU [CI]	P@0.5
On object (1,796)	73.7	92.3	3.79	84.4	36.0	39.7	57.9 [56.8, 59.0]	66.0
Near (599)	73.7	93.5	4.09	78.8	22.0	54.9	47.8 [45.8, 49.8]	49.2
Far (209)	72.7	92.8	4.16	80.9	29.0	36.8	29.8 [26.0, 33.6]	25.4
All (2,604)	73.7	92.6	3.89	82.8	32.2	43.0	53.3 [52.3, 54.3]	58.9

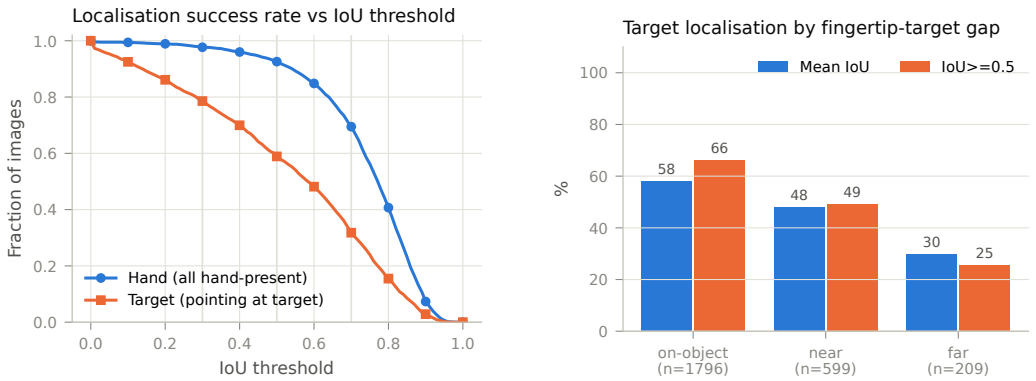


Fig. 4. Left: share of test images whose hand or target box reaches each IoU threshold. Right: target mIoU and P@0.5 for touched, near and far targets.

  - SpatialNet's P@0.5 (58.9%) and mIoU (53.3%) are above the zero-shot Qwen2.5-VL-7B (49.8%, 45.5%) and Qwen3-VL-8B (44.5%, 43.0%) reported by PointVG-R.
  - Those numbers average over all 3,072 test images, including 425 without a target. If those were scored as failures, Qwen2.5-VL-7B would reach about 57.8% P@0.5 and 52.8% mIoU on the images with a target, on a par with SpatialNet.
  - The zero-shot models draw tighter boxes: their P@0.7 (39.0–42.9%) exceeds SpatialNet's (31.8%).
  - SpatialNet finds the right region far more often: P@0.3 of 78.6% against 48.1–54.2%.
  - Against zero-shot models that are given the true hand box, it is slightly lower in P@0.5 (61.2–63.9%).

  We therefore claim parity at a fraction of the size, not superiority.
- *These comparisons are indicative only.* The published rows count IoU>τ, include no-hand test images scored in a way the papers do not state, and in one case use an unstated subset.
- *Cost.* VistaRef reports 22.9 images/s on a GPU, and the multimodal LLMs need billions of parameters. SpatialNet runs on the glasses in 60–80 ms per frame (Section 7.6).

7.2 Per-Head Accuracy and Target Distance

Table 5 breaks the result down by head and by fingertip–target distance, and Figure 4 shows the IoU success curves.

Table 6. The same 722,104-parameter model trained at three input resolutions (one seed each). Target metrics in %, on the 2,604 test images with a target. Fingertip error in % of the image width, so resolutions are comparable. CPU latency: PyTorch, 1 thread, on the Kaggle evaluation machine (reference only; it varies by about ±15% between runs). Bold: 95% paired-bootstrap interval of the difference from 160 px excludes 0.

Input	GFLOPs	CPU ms	P@0.5	P@0.7	mIoU	Far P@0.5	Fingertip (%)
160 px (main)	0.23	25.7	58.9	31.8	53.3	25.4	2.43
224 px	0.45	37.2	**62.4**	**37.1**	**54.8**	**35.9**	**1.81**
320 px	0.92	50.8	**65.1**	**40.3**	**56.5**	**34.9**	**1.32**

Hand presence. The hand gate separates no-hand from pointing frames with 98.6% accuracy (95% CI 98.1–98.9) and AUROC 0.998. It misses 15 of 2,604 hands (0.6%) and raises false alarms on 28 of 425 no-hand frames (6.6%). Choosing the threshold on validation so that no hand is missed (0.22) removes the misses but raises false alarms to 117 (27.5%), so we keep 0.5.

Where the errors come from.

- *Hand and fingertip are not the bottleneck.* The hand box and fingertip are equally good for all distances (hand P@0.5 of 92–94%; about 4 px fingertip error, 2.5% of the image side).
- *The target falls off with distance.* Target P@0.5 drops from 66.0% for touched targets to 49.2% near and 25.4% far (mIoU from 57.9 to 47.8 and 29.8).
- *The direction is coarse.* Its median error is 18.6°, and only 36.8% of far targets have an error below 15°. At a median fingertip–target distance of 0.32 image widths, 15° already moves the ray by about 0.08 image widths, the size of a small object. The error is largest for touched targets (36°), where the direction to the target centre is ill-defined and matters least.
- *Size is regressed towards the mean.* The predicted target centre lies inside the labelled box in 84.8% of images. Giving the model the true size with its own centre would raise P@0.5 to 70.4%; giving it the true centre with its own size would raise it to 72.3%. Errors in position and in extent therefore contribute about equally. Small targets (under 3% of the image) are predicted about 1.9 times too large, and reach a P@0.5 of only 36.9% (27.8–36.9% across seeds).

7.3 Input Resolution

Because a convolutional network shares its filters across positions, a larger input costs compute but no parameters. We retrain the unchanged model (722,104 parameters, same seed and settings) at higher input resolutions to test whether the input, rather than the model's capacity, limits localisation.

- *224 px.* The stored 224 × 224 images, used at full resolution instead of downsampled.
- *320 px.* The dataset re-converted from the original images at 320 × 320, so the larger input carries real detail, not upsampled pixels.

Results. Table 6 gives the results.

- *Resolution is the largest lever, at no parameter cost.*
  - From 160 to 320 px: P@0.5 +6.2 points (95% CI [+4.4, +8.0]), P@0.7 +8.5 [+6.8, +10.3], mIoU +3.2.
  - The fingertip error falls from 2.43% to 1.32% of the image width.
  - Each doubling of compute adds a steady 2.7–3.5 points of P@0.5.
  - These gains are far beyond retraining variation: across three seeds at 160 px, P@0.5 varies by 0.1 points and P@0.7 by 0.8.

- *Far targets gain first and then stop.* Far P@0.5 rises from 25.4% to 35.9% at 224 px (+10.5, CI [+4.3, +16.8]; the three 160 px seeds span 25.4–27.8%) and stays there at 320 px (34.9%). The direction error is flat at every resolution (31.4–32.2°). Beyond 224 px, far targets are limited by the direction estimate, not by the input.
- *Small objects need real image detail.* Targets under 3% of the image gain only at 320 px: 36.9% → 46.6% P@0.5 (+9.7, CI [+4.2, +15.2]). This is above all three 160 px seeds (27.8–36.9%), though small targets are the noisiest metric. Only the 320 px images were re-converted from the originals and carry detail that the stored 224 px images lack.
- *Shorter training, so a conservative estimate.* Both higher-resolution runs stopped early at 84 epochs (best epoch 59, against 100 at 160 px). Their gains are, if anything, underestimated. The validation loss is not comparable across resolutions, because the heatmap terms sum over more cells; this also makes early stopping trigger sooner.
- *Cost and choice of the deployed model.* Compute grows from 0.23 to 0.45 and 0.92 GFLOPs, while the weights stay at 1.6 MB. We deploy the 160 px model, the one measured on the glasses (Section 7.6). The 224 and 320 px results quantify how much accuracy is available if a device can afford two to four times the compute; we did not measure those two models on the glasses.

7.4 Ablations: What Each Component Buys

Each component of SpatialNet was added for a specific purpose. We report every ablation on all metrics (Table 7), but read each one first on the capability its component is meant to provide (Figure 5). We retrain the 160 px model without one component at a time, with the seed and settings of the main run:

- **A1** no distractor pasting and no ownership weighting;
- **A2** no geometric consistency terms ($\mathcal{L}_{\text{ray}}$ and the hand-axis term);
- **A3** no ray conditioning of the target head (no G^t; 13,105 fewer parameters);
- **A4** no instance-anchored chain: the direction head and gates read the globally pooled vector, and the fingertip has no hand-anchored terms. This gives a plain model of the same size (709K parameters).

Each ablation was trained once, and we test each difference in two ways. A paired bootstrap over test images (resampling the images and recomputing the difference on the same images) gives a 95% confidence interval for test-sample variation. Retraining the full model with two more seeds gives the training variation: a difference counts only if it exceeds the largest change that retraining alone produced (Table 7, second row).

No component costs coarse accuracy. All four variants stay within ±1.1 points of the full model's P@0.5 (58.9%), and none differs significantly. Each component therefore adds a specific capability on top of the same coarse localisation, at a cost of at most 13K parameters. The capabilities follow.

A4, instance anchoring: precision in a small model. Against a plain model of the same size, the anchored chain improves everything that requires precision:

- P@0.7: +2.8 [+1.1, +4.3], against at most 0.8 between seeds;
- hand mIoU: +1.8 [+1.4, +2.2], against at most 0.3;
- direction error: 3.4° lower [2.4, 4.5], against at most 1.2°;
- small targets: +11.3 points [+6.5, +16.5]. The plain model's 25.6% is below all three seeds of the full model (27.8–36.9%), and its small boxes are drawn 2.4 instead of 1.9 times too large.

A plain model of the same size thus finds the right region as often (P@0.5 unchanged), but cannot draw precise boxes or handle small objects. That is the role of the anchoring. A4 stopped training

Table 7. Ablations at 160 px (one seed each). First row: the full model's values; other rows: difference from the full model (points, or degrees for the direction error, where positive means worse). *Retraining*: the largest difference of seeds 1 and 2 from seed 42, i.e. what retraining the full model alone changes. Bold: the 95% paired-bootstrap interval excludes 0 and the difference exceeds the retraining row. *Small*: targets under 3% of the image (309 images). *With other hands*: test images with one or two pasted hands (Section 7.5).

	Clean test images							With other hands	
Variant	mIoU	P@0.5	P@0.7	Far	Small	Hand	Dir. (°)	P@0.5	No hand
Full model	53.3	58.9	31.8	25.4	36.9	73.7	32.2	60.1	91.8
Retraining (seeds 1, 2)	±0.9	±0.1	±0.8	±2.4	±9.1	±0.3	±1.2	±0.4	±3.3
A1 no hand pasting	**−1.2**	−0.1	−1.3	+1.9	−3.9	**−0.6**	**+1.8**	**−4.1**	**−5.4**
A2 no ray / axis losses	−0.5	+1.1	+0.6	+3.3	−0.6	−0.4	**+1.5**	0.0	−1.2
A3 no ray conditioning	+0.2	+0.7	**+1.9**	+5.3	+1.0	0.0	**−1.3**	+1.0	+1.2
A4 no anchor chain	**−1.4**	−0.3	**−2.8**	+0.5	**−11.3**	**−1.8**	**+3.4**	−0.5	0.0

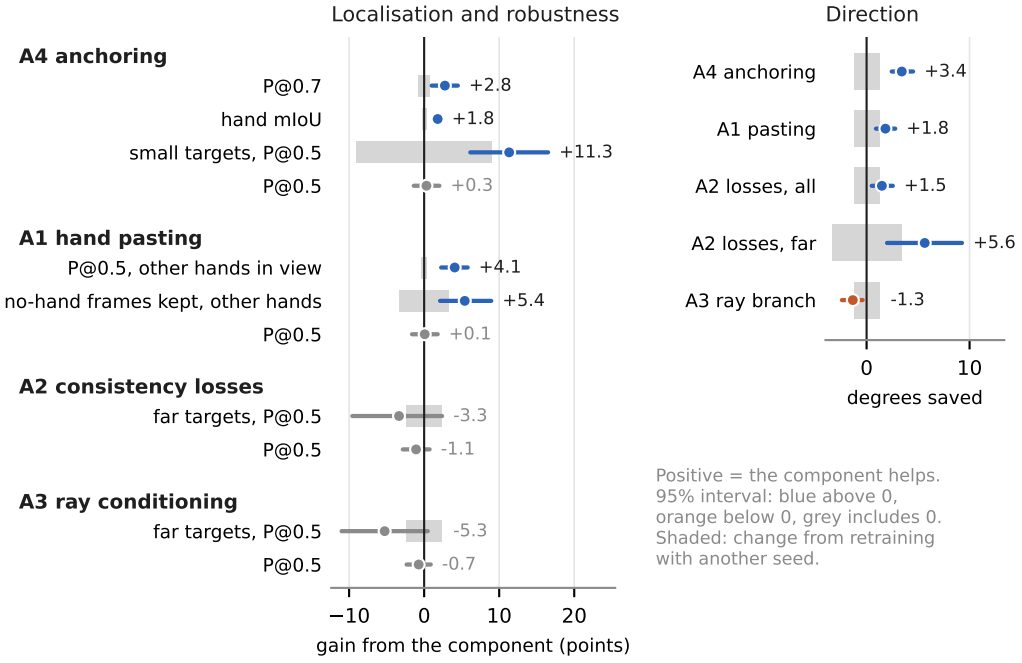


Fig. 5. What each component contributes, with 95% paired-bootstrap intervals. Gain = full model minus ablation, or degrees of direction error saved, so positive means the component helps. Grey bands: the largest change that retraining the full model with another seed produced on the same metric. For each component, the first rows show the capability it was designed for and the last row shows P@0.5.

earliest (best epoch 54, against 100). Shorter training could widen its gaps, but it would be expected to lower P@0.5 as well; instead, the losses are confined to the precision metrics.

A1, distractor pasting: robustness to other people's hands. With other hands in view, the model trained with pasting:

- keeps 4.1 more points of target P@0.5 [+2.4, +5.8];
- keeps 5.4 points more no-hand frames as *no hand* [+1.9, +8.9];
- puts its hand on another person's hand six times less often (0.2% against 1.2%).

On clean images it costs nothing (P@0.5 −0.1), and it even helps slightly: mIoU +1.2, hand mIoU +0.6, and 1.8° less direction error. A plausible reading is that pasting acts as a regulariser, making the model rely on the hand's position and context rather than its appearance alone.

The ownership-accuracy metric itself barely moves (99.7% against 98.7%), because pasted hands sit higher in the frame than the wearer's (Section 7.5). The informative measures are the two above. The pasted test hands follow the training procedure; real other people's hands are tested separately in Section 7.5.

A2, consistency losses: a sharper pointing direction. The direction is one of SpatialNet's four outputs, and an interface can use it directly, for example to draw the pointing ray as feedback.

- Without the losses, the far-target direction error rises by 5.6° [+2.1, +9.4], to 34.6°, above all three seeds of the full model (29.0–32.3°). This is where the ray-alignment term weighs most.
- Overall, the direction error rises by 1.5°, only slightly more than retraining alone changes it (1.2°).
- The losses do not change target-box accuracy (+1.1 P@0.5, n.s.), because the target head does not draw on the predicted direction (A3 below).

A3, explicit ray geometry: what a small model learns from it. We gave the target head the predicted ray as explicit geometry and asked whether a small model uses it to find far targets. Removing the branch G^t changes P@0.5 by less than one point (+0.7, n.s.). P@0.7 is 1.9 points higher [+0.2, +3.6] and the direction error 1.3° lower; both lie outside the seed spread (P@0.7 31.0–31.8%, direction 32.2–33.5°), so the branch, as trained, costs a little precision, and A3 also trained 20% longer (best epoch 120). Probing the trained full model on 500–800 test images explains this:

- *The model relies on the branch.* Its output is as large as the visual term (median ratio of spreads 0.86), and switching it off at inference costs 9.0 points of P@0.5.
- *It has learned proximity, not direction.* The branch's output correlates with closeness to the fingertip ($\rho = -0.35$) and to the ray's line (−0.33), but hardly with being in front of the fingertip (+0.04). The same holds for far targets.
- *So it helps only near the finger.* Switching it off costs 11 points of P@0.5 on touched targets and 9.5 on near ones, and none on far ones.

With 70% of targets touched and 7% far, the training signal rewards a "near the fingertip" prior. The visual head can learn that prior by itself, so a model trained without the branch reaches the same accuracy.

The branch can therefore be dropped without loss. We keep it in the deployed model because that is the model measured on the glasses, and because it is the place where a forward-constrained ray prior would enter.

The finding carries over to other pointing models: on touch-dominated data, giving a model the ray is not enough. The ray must be constrained to point forward, or far targets must be weighted up in training, before it can help them. Together with A2 and the flat direction error across resolutions (Section 7.3), it locates the remaining bottleneck for far targets in how the direction is estimated and used, not in how the ray is encoded.

Robustness of these conclusions.

Table 8. Ownership on test images with one or two pasted hands (synthetic), and on real frames with other people's hands (T5). All values in %. *On other hand*: predicted hand centre inside a pasted hand. *No hand kept*: no-hand images still classed as no hand.

	Ownership accuracy	On other hand	No hand kept	Target P@0.5
Full model, clean test images	–	–	93.4	58.9
Full model, pasted hands	99.7 [99.4, 99.9]	0.2	91.8	60.1
A1 (no pasting), pasted hands	98.7 [98.2, 99.1]	1.2	86.4	56.1
Full model, real frames ([N])	xx.x [xx.x, xx.x]	xx.x	–	–

- Each ablation is a single run. Every difference we report as an effect is significant under the paired bootstrap *and* larger than the change produced by retraining the full model with another seed (Table 7).
- The effects differ in margin. The anchoring's P@0.7, hand and direction gains and pasting's other-hands gains are several times the retraining spread, and A2's far-target direction gain is well beyond it. The small-target gain of anchoring (11.3 against 9.1) and the overall direction changes of A1–A3 (1.3–1.8° against 1.2°) are close to it, and we treat them as supporting evidence only.
- We draw no conclusions from P@0.5 differences: retraining moves it by 0.1 points, and no ablation moves it by more than 1.1.
- The loss weights were not searched (Appendix A). A2 covers the two non-standard terms, and the remaining eleven are standard box, keypoint, heatmap and focal losses.

7.5 Following the Wearer's Hand

Synthetic distractors. With one or two pasted hands in each test image, the model's hand still lies nearer the wearer's hand than any pasted hand in 99.7% of hand images (Table 8), and other metrics barely change. This is weaker evidence than it looks:

- The pasted hands follow the training distribution.
- The ownership metric is easy under that distribution. In the first training epoch, when the hand IoU was only 0.38, ownership accuracy on training batches was already 93%, because pasted hands sit in the upper part of the frame and the wearer's hand in the lower part.

The more informative number is that pasted hands rarely turn a no-hand frame into a hand frame: 91.8% of no-hand frames stay *no hand*, against 93.4% without pasted hands. Ablation A1 (Section 7.4) shows that the training recipe is needed: without it, target P@0.5 with other hands in view drops by 4.1 points and no-hand frames kept drop by 5.4, while clean accuracy is unchanged. Robustness to real other people's hands is tested by the real-frame check below.

Real frames. [Fill in when T5 is done:]

- We recorded [N] clips with the glasses in which another person's hand is visible, sometimes pointing, with [consent procedure], and sampled [100–150] frames.
- Two annotators independently labelled the predicted hand in each frame as the wearer's, another person's, or missed.
- We report ownership accuracy with a Wilson interval and inter-annotator agreement (Cohen's κ).

Figure 6 shows examples.

Six real frames from the glasses with another person's hand in view: predicted wearer's hand, fingertip, ray and target (T5). Include at least one failure.

Fig. 6. Real frames with other people's hands. [Insert after T5.]

Table 9. Inference cost of the 160 px model. Server rows: PyTorch eager mode, batch size 1, on the 4-core Kaggle virtual machine used for evaluation (reference only; at this model size they are dominated by framework overhead). Glasses rows: the engineering unit of Section 6.2. * Derived from the per-frame latency, assuming frames are processed back to back.

Platform and measure	Value
Server CPU, 1 thread: latency, median / p90	25.7 / 28.0 ms
Server CPU, 4 threads: latency, median / p90	25.8 / 27.0 ms
Server GPU, NVIDIA Tesla T4: latency, median / p90	15.9 / 16.3 ms
Glasses: inference latency per frame	60–80 ms
Glasses: throughput*	12–17 frames/s
Glasses: peak additional RAM during inference	≈100 KB
Glasses: battery drain per minute, SpatialNet running continuously	9%
Glasses: battery drain per minute, streaming frames to the phone at 0.5 fps	16%

7.6 On-Device Performance

Table 9 reports cost on the evaluation machine and on the glasses.

Results.

- *Latency.* SpatialNet takes 60–80 ms per frame on the glasses. That is within a 100–200 ms budget for interactive response, and allows 12–17 frames per second. The reference GPU model, VistaRef, reports 22.9 images per second, but on a server GPU and with over 300 times as many parameters.
- *Memory.* Inference needs about 100 KB of RAM beyond the loaded model, and the FP16 weights take 1.6 MB. Running the weights in half precision gives the same target IoU as FP32 on 320 validation images (0.510 in both), so halving the model size costs no accuracy.
- *Battery: on-device processing is cheaper than offloading.* Running SpatialNet continuously drains 9% of the battery per minute. Merely streaming frames to the phone at 0.5 fps, so that the work could be done elsewhere, drains 16%. That is almost twice as much, even before counting the phone's or a server's own compute.
- *The comparison favours offloading, and on-device still wins.* The streaming mode sends only one frame every two seconds, while SpatialNet processes frames continuously.
- *Absolute drain.* The absolute figures reflect an engineering device without production power management, and a one-minute measurement with 1% resolution. They support the comparison between modes, not a battery-life estimate. Continuous inference is also the worst case: in use, the model would run only while the wearer is pointing.

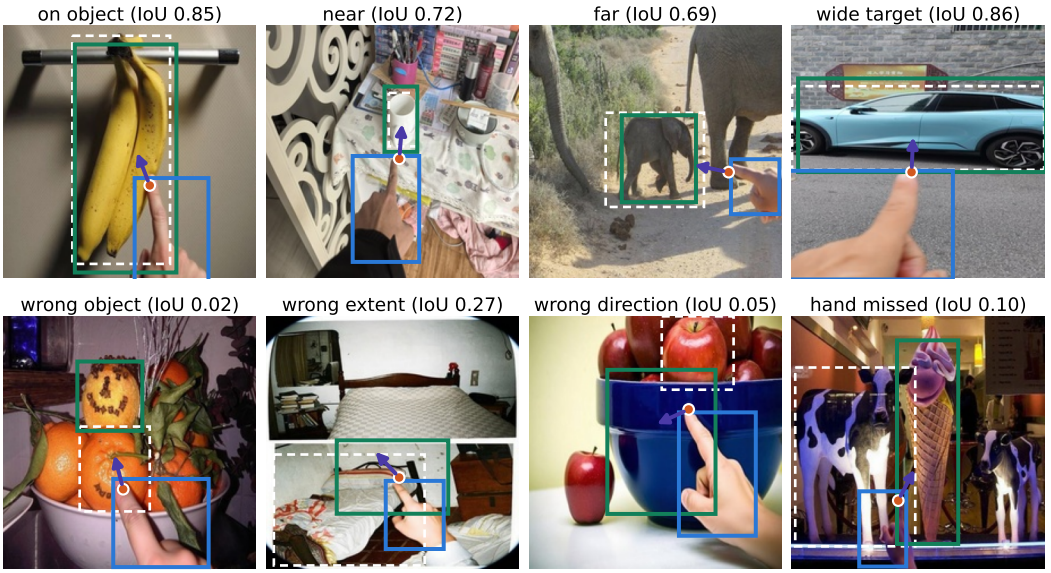


Fig. 7. Successes (top) and failures (bottom) on EgoPoint test images: predicted hand (blue), fingertip and direction, and target (green); labelled target dashed. Bottom row, one example per failure type of Section 7.7.

- *Higher resolutions.* The 224 and 320 px models were not measured on the glasses. Their compute is two and four times that of the deployed model, so they trade latency and power for the accuracy gains of Section 7.3.

7.7 Qualitative Results and Failure Analysis

Figure 7 shows typical successes and failures. We sorted the 558 test images with target IoU < 0.3 (21.4%) by rule:

- *Right object, wrong extent* (36%): the predicted centre lies inside the labelled box. These are mostly touched targets (137 of 199), for example part of a bed instead of the whole bed.
- *Wrong direction* (33%): direction error over 45°.
- *Wrong object despite a reasonable direction* (31%): mostly near and far targets (130 of 171), typically a similar object next to the labelled one along the ray.
- *Hand missed* (1%).

Some labelled targets are ambiguous: in the "wrong direction" example, the finger touches the bowl but the label is an apple above it. Such cases count as failures in every metric, so they lower our numbers and those of all compared models alike.

8 Discussion

What a single-image, single-pass CNN can and cannot do. The results separate the task into two parts.

- *Easy at this size.* Finding the wearer's hand, the fingertip and whether a hand is present works well: hand P@0.5 of 92.6%, 3.9 px fingertip error, and AUROC 0.998.
- *Hard at this size.* Deciding which object the finger refers to, and how far that object extends, is what the large models do much better. Our errors split about evenly between wrong extent, wrong direction and wrong object (Section 7.7).

A 0.72M CNN at 160 × 160 therefore makes a good always-on front end: it can trigger and crop for a heavier model, but should not be expected to reach large-model accuracy alone.

What matters at this scale. The ablations and the resolution study rank the levers for a sub-million-parameter pointing model.

(1) *Input resolution.* It gives the largest gains (P@0.5 +6.2, far targets +10.5) at no parameter cost, only compute. For a device that can afford it, resolution is the first knob to turn.
(2) *Instance anchoring.* It is what keeps boxes precise (P@0.7, small targets, hand box, direction). A plain model of the same size reaches the same P@0.5, but not the same precision.
(3) *Distractor pasting.* It is cheap robustness to other people's hands, with no cost on clean images.
(4) *Explicit ray geometry.* As used here, it is not a lever: the learned prior settles on fingertip proximity. Far targets need either a better direction estimate or a ray prior that is constrained to point forward.

Why most targets are touched, and what that implies. Where people point at distant objects depends on the eye as well as the arm: modelling the dominant eye makes pointing rays markedly more accurate [30], and common ray-casting models show systematic offsets [27]. If wearers align the fingertip with the target from the eye, a camera next to the eye sees the target at or just behind the fingertip. This is consistent with about 70% of EgoPoint targets containing the fingertip in the image, although part of EgoPoint is staged or generated [confirm]. It has two consequences:

- For such targets, the fingertip tells *where* the object is but not *how large* it is: a mean-size box at the true fingertip reaches a P@0.5 of only 11.4% (Table 4). On touched objects, the target head mainly has to estimate the object's extent.
- The ray matters for the remaining near and far targets, where the finger points away from the line of sight. This subset is small (7% far), so it needs its own metrics rather than pooled averages. It is also too small to shape a learned ray prior: our prior learned fingertip proximity, not direction (Section 7.4).

Limitations.

- *Two scenes.* The pointing and target gates were never trained or tested against negatives (Section 5). In deployment, a non-pointing hand may therefore be read as pointing, and the cascade's middle states are untested. A dataset with non-pointing hands and pointing without a target is needed before relying on them.
- *Ownership.* Our distractors are synthetic and encode the prior that the wearer's arm enters from the bottom of the view. The real-frame check (Section 7.5) is small. A wearer pointing upward from a raised arm, or a companion's hand close to the wearer's, can break the prior.
- *Target extent from a local window.* The target size is regressed from a 3 × 3 neighbourhood of the stride-8 map around the predicted centre.
  - At 160 px, P@0.7 is 31.8%, and small targets are drawn about twice too large.
  - Both improve with anchoring (A4) and with real image detail (320 px: P@0.7 40.3%, small targets 46.6%).
  - Small boxes remain about 1.6 times too large even at 320 px, so sampling the size from a box-shaped region around the first estimate is a natural next step.
- *A coarse direction.* The direction error (median 18.6°) does not improve with resolution, and A2 shows that the consistency losses improve it but that the target head does not exploit it. Far targets level off near 35% P@0.5. A direction supervised by the finger's own axis (for example from root and fingertip keypoints), and a forward-constrained ray prior, are the most promising remedies.

- *A 2D ray.* The ray lives in the image plane. Objects at different depths along the same image ray are told apart only by appearance, and the eye–camera offset is ignored. Depth, or a second view, could resolve such cases.
- *Single frames.* The model has no temporal context. Detecting a pointing *event*, as opposed to a pointing frame, and smoothing the target over a short window are left to the application.
- *Dataset coverage.* Results come from one dataset with mostly landscape, partly synthetic images and unknown demographic coverage (for example, skin tone and handedness). Flip augmentation covers left hands only geometrically. Evaluation on real, diverse glasses footage is the most important next step.
- *Interaction.* We measure the model on a benchmark and on the device, not the interaction. How wearers respond to its errors, and which confirmation designs suit them, need a study with participants wearing the glasses; we leave it to future work.
- *Baselines.* We copied the published state-of-the-art results instead of re-running those models, and two of them use different inputs or subsets, or average over test images without a target. We did not re-run any multimodal LLM under our exact protocol. Large grounding models are clearly more accurate. Our claim is the operating point, with accuracy reported next to its cost, not superiority.

Implications for glasses interfaces. A model of this size can run continuously and act as a gate for more expensive components:

- *Crops instead of frames.* Only when the glasses detect a pointing frame with a confident target does a recogniser or assistant run, and then on the target crop instead of the full frame. This reduces computation, bandwidth and the amount of bystander imagery that leaves the device. Our measurements support this: on the test unit, running SpatialNet continuously drains less battery (9% per minute) than streaming frames to the phone at only 0.5 fps (16%).
- *Confirmation for far targets.* Because far targets are where errors concentrate (Section 7.2), interfaces should confirm the selection, for example by naming the object or highlighting it, before acting on it.

9 Ethics and Privacy

Bystanders. A head-worn camera sees people who did not consent to being recorded.

- SpatialNet runs on the glasses, so localisation needs no frame to leave the device.
- A downstream step can be limited to the target crop and to moments when the wearer points.
- SpatialNet is trained to ignore other people's hands. It does not detect, identify or track people.

On-device processing still does not remove the need for visible recording indicators; the test unit has user- and bystander-facing LEDs.

Data. EgoPoint is used under the access terms of its authors' release, for research only. We do not redistribute its images or our converted labels; our conversion script regenerates the labels from the original release. The images shown in this paper come from its test and training splits. [T5: describe consent for the real frames, the ethics approval or exemption, data retention, and the avoidance of bystanders' faces.]

Availability. We will release the code, the dataset conversion script, the trained weights and the per-image evaluation outputs (from which every table can be recomputed) upon publication.

10 Conclusion

SpatialNet localises the object a smart-glasses wearer points at from nothing but one RGB frame. It uses one forward pass of a 0.72M-parameter CNN, with no prompt, language model or external pose model. It does so by chaining instance-anchored heads (hand, fingertip, direction, target), which keep its boxes precise at this size. Our experiments show that input resolution is the largest further lever, and that a learned ray prior collapses into a fingertip-proximity prior when most targets are touched. Training with pasted, unlabelled hands teaches it to follow the wearer's hand. It finds the wearer's hand and fingertip reliably and the pointed-at object with a P@0.5 of 58.9% (mIoU 53.3%), but only 25.4% when the object is far from the fingertip. That is well below large GPU models, but it runs on the glasses in 60–80 ms per frame and costs less battery than streaming frames to a phone. The main open problems are pointing scenes that EgoPoint does not contain, real bystander hands, and far targets along ambiguous rays. We expect progress on them to come from richer egocentric pointing data at least as much as from larger models.

References

[1] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2.5-VL Technical Report. arXiv preprint arXiv:2502.13923.

[2] Sven Bambach, Stefan Lee, David J. Crandall, and Chen Yu. 2015. Lending A Hand: Detecting Hands and Recognizing Activities in Complex Egocentric Interactions. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 1949–1957.

[3] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. 2015. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 28. 1171–1179.

[4] Richard A. Bolt. 1980. "Put-That-There": Voice and Gesture at the Graphics Interface. In *Proceedings of the 7th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '80)*. ACM, 262–270.

[5] Yixin Chen, Qing Li, Deqian Kong, Yik Lun Kei, Song-Chun Zhu, Tao Gao, Yixin Zhu, and Siyuan Huang. 2021. YouRefIt: Embodied Reference Understanding with Language and Gesture. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

[6] Yura Choi, Roy Miles, Rolandos Alexandros Potamias, Ismail Elezi, Jiankang Deng, and Stefanos Zafeiriou. 2026. Do You See What I Am Pointing At? Gesture-Based Egocentric Video Question Answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[7] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A Large-Scale Hierarchical Image Database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 248–255.

[8] Debidatta Dwibedi, Ishan Misra, and Martial Hebert. 2017. Cut, Paste and Learn: Surprisingly Easy Synthesis for Instance Detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 1301–1310.

[9] Bradley Efron and Robert J. Tibshirani. 1994. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.

[10] Zhen-Hua Feng, Josef Kittler, Muhammad Awais, Patrik Huber, and Xiao-Jun Wu. 2018. Wing Loss for Robust Facial Landmark Localisation with Convolutional Neural Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2235–2245.

[11] Zhora Gevorgyan. 2022. SIoU Loss: More Powerful Learning for Bounding Box Regression. arXiv preprint arXiv:2205.12740.

[12] Golnaz Ghiasi, Yin Cui, Aravind Srinivas, Rui Qian, Tsung-Yi Lin, Ekin D. Cubuk, Quoc V. Le, and Barret Zoph. 2021. Simple Copy-Paste Is a Strong Data Augmentation Method for Instance Segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2918–2928.

[13] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. 2017. Mask R-CNN. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2961–2969.

[14] Yichao Huang, Xiaorui Liu, Xin Zhang, and Lianwen Jin. 2016. A Pointing Gesture Based Egocentric Interaction System: Dataset, Approach and Application. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*.

[15] Varun Jain, Gaurav Garg, Ramakrishna Perla, and Ramya Hebbalaguppe. 2019. GestARLite: An On-Device Pointing Finger Based Gestural Interface for Smartphones and Video See-Through Head-Mounts. AAAI 2019 Workshop on Plan, Activity, and Intent Recognition. arXiv:1904.09843.

[16] Hei Law and Jia Deng. 2018. CornerNet: Detecting Objects as Paired Keypoints. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 734–750.

[17] Jaewook Lee, Jun Wang, Elizabeth Brown, Liam Chu, Sebastian S. Rodriguez, and Jon E. Froehlich. 2024. GazePointAR: A Context-Aware Multimodal Voice Assistant for Pronoun Disambiguation in Wearable Augmented Reality. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. ACM, Article 408. doi:10.1145/3613904.3642230

[18] Chentao Li, Zirui Gao, Mingze Gao, Yinglian Ren, Jianjiang Feng, and Jie Zhou. 2026. Do MLLMs Understand Pointing? Benchmarking and Enhancing Referential Reasoning in Egocentric Vision. arXiv preprint arXiv:2604.21461.

[19] Ling Li, Zhizhen Cai, Xinkun Wu, Ziyu Zhu, Jiaqing Lyu, Bowen Liu, and Zhidong Deng. 2026. VistaRef: Boosting Visual Spatial Orientation Awareness for Pointing-to-Object Detection. arXiv preprint arXiv:2606.24498.

[20] Ling Li, Bowen Liu, Zinuo Zhan, Peng Jie, Jianhui Zhong, Kenglun Chang, and Zhidong Deng. 2026. Beyond Language: Grounding Referring Expressions with Hand Pointing in Egocentric Vision. arXiv preprint arXiv:2603.26646.

[21] Ling Li, Bowen Liu, Zinuo Zhan, Jianhui Zhong, Ziyu Zhu, Bingcai Wei, Kenglun Chang, and Zhidong Deng. 2026. PointVG-R: Internalizing Geometric Reasoning in MLLMs for Precise Pointing Localization via Visual Chain of Thought. arXiv preprint arXiv:2606.24539.

[22] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal Loss for Dense Object Detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2980–2988.

[23] Rosanne Liu, Joel Lehman, Piero Molino, Felipe Petroski Such, Eric Frank, Alex Sergeev, and Jason Yosinski. 2018. An Intriguing Failing of Convolutional Neural Networks and the CoordConv Solution. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 31.

[24] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. 2024. Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. In *Proceedings of the European Conference on Computer Vision (ECCV)*.

[25] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations (ICLR)*.

[26] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2018. ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 116–131.

[27] Sven Mayer, Katrin Wolf, Stefan Schneegass, and Niels Henze. 2015. Modeling Distant Pointing for Compensating Systematic Displacements. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15)*. ACM, 4165–4168. doi:10.1145/2702123.2702332

[28] Shu Nakamura, Yasutomo Kawanishi, Shohei Nobuhara, and Ko Nishino. 2023. DeePoint: Visual Pointing Recognition and Direction Estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 20577–20587.

[29] Zhiliang Peng, Wenhui Wang, Li Dong, Yaru Hao, Shaohan Huang, Shuming Ma, and Furu Wei. 2023. Kosmos-2: Grounding Multimodal Large Language Models to the World. arXiv preprint arXiv:2306.14824.

[30] Katrin Plaumann, Matthias Weing, Christian Winkler, Michael Müller, and Enrico Rukzio. 2018. Towards Accurate Cursorless Pointing: The Effects of Ocular Dominance and Handedness. *Personal and Ubiquitous Computing* 22 (2018). doi:10.1007/s00779-017-1100-7

[31] Dandan Shan, Jiaqi Geng, Michelle Shu, and David F. Fouhey. 2020. Understanding Human Hands in Contact at Internet Scale. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 9869–9878.

[32] Leslie N. Smith and Nicholay Topin. 2019. Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications (Proceedings of SPIE, Vol. 11006)*.

[33] Xiao Sun, Bin Xiao, Fangyin Wei, Shuang Liang, and Yichen Wei. 2018. Integral Human Pose Regression. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 529–545.

[34] Mingxing Tan, Ruoming Pang, and Quoc V. Le. 2020. EfficientDet: Scalable and Efficient Object Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 10781–10790.

[35] Asher Trockman and J. Zico Kolter. 2023. Patches Are All You Need? *Transactions on Machine Learning Research* (2023).

[36] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212.

[37] Lingxiao Yang, Ru-Yuan Zhang, Lida Li, and Xiaohua Xie. 2021. SimAM: A Simple, Parameter-Free Attention Module for Convolutional Neural Networks. In *Proceedings of the 38th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 139)*. 11863–11874.

[38] Haoxuan You, Haotian Zhang, Zhe Gan, Xianzhi Du, Bowen Zhang, Zirui Wang, Liangliang Cao, Shih-Fu Chang, and Yinfei Yang. 2024. Ferret: Refer and Ground Anything Anywhere at Any Granularity. In *International Conference on Learning Representations (ICLR)*.

[39] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. 2018. mixup: Beyond Empirical Risk Minimization. In *International Conference on Learning Representations (ICLR)*.

[40] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. 2020. Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 12993–13000.
[41] Xingyi Zhou, Dequan Wang, and Philipp Krähenbühl. 2019. Objects as Points. arXiv preprint arXiv:1904.07850.

A Loss Weights

Table 10 lists the 13 loss terms and their weights. The weights were set by hand during development, using the training and validation splits only, and were not tuned by a search. Ablation A2 (Section 7.4) removes the two non-standard terms; the others are standard box, keypoint, heatmap and focal losses.

Table 10. Loss terms, their supervision, and weights.

Term	Loss	Supervision	Weight
Hand box	$1 - \mathrm{CIoU}$	hand box	0.10
Fingertip	Wing (w=0.5, ϵ=0.1)	fingertip	0.18
Direction	$1 - \cos$ine	fingertip $\rightarrow$ target centre	0.03
Target box	$1 - \mathrm{SIoU}$	target box	0.45
Target box, auxiliary	$1 - \mathrm{IoU}$	target box	0.10
Hand gate	focal BCE, positives $\times 3$	H	0.08
Pointing gate	focal BCE, positives $\times 2$	P (hand images)	0.10
Target gate	focal BCE, positives $\times 2$	T (pointing images)	0.06
Ray alignment	distance to ray + $0.5 \cdot$ behind	none (predictions only)	0.08
Hand axis	hinge outside a 60° cone	none (predictions only)	0.02
Hand heatmap	penalty-reduced focal, ownership-weighted	hand centre	0.10
Fingertip heatmap	penalty-reduced focal	fingertip	0.10
Target heatmap	penalty-reduced focal	target centre	0.10

B Augmentation Parameters

Table 11. Training-time augmentation, applied on the fly; each training image is seen once per epoch as a new random variant. Validation and test images are only resized and normalised.

Step	Setting
Distractor hands	1–2 hands with probability 0.5 (hand images) or 0.3 (no-hand images); scale 0.45–1.0; rotation by 90/180/270° and optional mirror with probability 0.7; centre in the top 65% of the frame; overlap with the wearer's hand and the target below 15%; feathered alpha (12% of the crop size); colour jitter; 30% self-paste; pool of 512 donor crops, half from pointing hands
Decoy patch	unlabelled non-hand patch from a no-hand image, with probability 0.15
Horizontal flip	0.5
Shift / scale / rotate	±5% / ±10% / ±15°, probability 0.4
Perspective	scale 0.02–0.05, probability 0.2
Brightness / contrast	±20%, probability 0.5
Gaussian blur	kernel 3–5, probability 0.2
Mixup	within the same scene, $\lambda \sim \mathrm{Beta}(0.2, 0.2)$ folded to ≥ 0.5, label of the dominant image, every batch
Anchor forcing	ground-truth anchors with probability 0.5 decaying to 0 over 20 epochs